

Revenue Plan

How this product makes money after removing AdSense — options, arithmetic, sequencing, and an implementation path.

Prepared for	Bhargav (bhargavgk00@gmail.com)
Date	20 August 2026
Product	Yogatik v3.10.2 — browser-native AI assistant, plus Electron/Tauri desktop
Status	AdSense integration removed from the codebase; no monetization active

Read this first

Every model in this document is downstream of one number you have not shared: monthly active users. The recommendations assume a small base today, which is consistent with the AdSense “low value content” rejection. If your real userbase is materially larger, go to Section 3 and re-rank accordingly.

1. Where things stand

The AdSense route is closed, and it was never really open. Google flagged **yogatik.firebaseio.com** with two violations: *Google-served ads on screens without publisher-content*, and *Low value content*. The first was caused by a self-built full-screen interstitial that fired every tenth chat and contained nothing but an ad slot, a countdown and a Continue button. The second is structural: Yogatik is a single-page app whose served HTML is an empty root element, so a crawler sees no content at all.

Both are resolved by removal. The following was deleted from the codebase on 19 August 2026:

- **AdModal.jsx** — the interstitial component, the hardcoded publisher ID and slot, and the placeholder panel that sat behind the ad unit.
- **index.html** — the `adsbygoogle.js` script tag.
- **public/ads.txt** — the publisher authorization file.
- **App.jsx** — the every-tenth-chat trigger, the `showAd` state, the `chatCountRef` counter, and the overlay term in the browser-panel occlusion check.
- **styles.css** — the modal animation rules.

Verified after removal: `npm run lint` reports 0 errors, and `npm test` passes 1,232 tests across 95 files.

What the product actually is

This matters more than any monetization tactic, because it determines which models are even available. Yogatik is not a content site and not a hosted SaaS. It is a local-first client.

Property	Consequence for revenue
Zero backend; all state in IndexedDB	No per-user server cost, so margins are near-total on anything you sell. But there is also no usage meter to bill against.
Bring-your-own API key	You carry none of the inference cost. You also capture none of the spend, and key setup is the single largest drop-off point in onboarding.
Firebase auth and Firestore, encrypted key sync	The only server-side surface you already run and pay for. It is the natural foundation for a paid sync or teams tier.
Desktop build with shell, filesystem, computer control and real browser automation	The genuine moat. None of it is reproducible in a web app by any competitor. This is the most defensible thing to charge for.
Privacy-first positioning: nothing leaves the device	Rules out every data-brokerage and telemetry-monetization model, not merely on ethics but because the claim is the product.

The real constraint

The AdSense rejection is a symptom, not the disease. A site with meaningful traffic and content would have been approved. Monetization design is downstream of distribution: no model below produces meaningful revenue without users, and two of the strongest options need almost none.

2. The arithmetic that decides the strategy

The question “can I earn a little from many users?” has a numeric answer. Micro-monetization on a tool audience realistically yields **\$0.02 to \$0.10 per monthly active user**. Direct sales yield **\$0.30 to \$1.00 per monthly active user** at ordinary conversion rates. The crossover is not close.

Model A — pennies at volume (ads, affiliate, search revenue share)

Monthly active users	At \$0.02 each	At \$0.06 each	Verdict
1,000	\$20	\$60	Not worth the engineering time
10,000	\$200	\$600	Covers hosting, not effort
50,000	\$1,000	\$3,000	Begins to matter
500,000	\$10,000	\$30,000	A real business

Model B — direct sales (paid desktop tier at \$39 one-time)

Monthly active users	At 1% conversion	At 2.5% conversion	Verdict
1,000	\$390	\$975	Beats every penny model by 6 to 15 times
10,000	\$3,900	\$9,750	Meaningful income
50,000	\$19,500	\$48,750	Full-time viable

Conversion figures are illustrative industry ranges for developer-tool desktop software, not measurements of your product. Treat them as an order-of-magnitude comparison between models, not as a forecast. The structural conclusion holds across the whole plausible range: **at any userbase you are likely to have in the next twelve months, per-user dollars beat per-user pennies by an order of magnitude.**

Strategic implication

Build direct-sale revenue first, at small scale, because it works at small scale. Add penny-scale models later, only once distribution work has produced the volume that makes them non-trivial. Doing it in the opposite order, which is what the ad interstitial was, costs user goodwill and earns nothing.

3. Revenue options, ranked

Each option is scored on fit with the existing architecture, engineering effort, and the userbase required before it is worth building at all.

#	Model	Fit	Effort	Needs users?	Revenue shape
1	Paid desktop tier	Very high	Medium	No	\$29–49 one-time, or \$5–7 monthly
2	Sync and teams subscription	High	Medium	No	\$4–8 monthly, recurring
3	B2B / self-host license	High	High	No	\$1k–20k per contract
4	Managed keys / credits	Medium	High	Some	20–30% margin on usage
5	Plugin marketplace cut	Medium	High	Yes	20–30% of third-party sales
6	Search partner revenue	Medium	Low	Yes	Per-query micro-payments
7	Store distribution	Medium	Low	No	Paid listing plus discovery
8	Sponsorship / tips / affiliate	Low	Very low	Some	Irregular, small

3.1 Paid desktop tier — the primary recommendation

The web build is a good chat UI, and many of those exist. The desktop build holds `terminal_run`, the per-chat working-folder filesystem, `computer_control`, real browser automation through `WebContentsView`, clipboard history, folder watching, OS keychain sealing, and the floating companion. A browser-based competitor cannot ship any of it. That asymmetry is what you sell.

- **Model:** free web app, paid desktop. The free desktop tier stays fully usable for chat; the OS-integration tools are the paid surface.
- **Why one-time pricing:** desktop utilities convert far better on a single purchase than on a subscription. Offer “\$39, includes twelve months of updates” — perpetual use of what was bought, renewal optional.
- **Infrastructure you already have:** electron-builder, GitHub Releases, and a working electron-updater pipeline. The only missing piece is license validation.
- **Infrastructure you need:** a merchant of record to issue keys and handle tax. See Section 5.3.

3.2 Sync and teams subscription

You already run Firebase auth and Firestore, and `firestore.rules` already scopes documents per user ID. Extending that from encrypted API keys to conversations, skills, agents and plugin sets is incremental work on infrastructure you are already paying for. Sell it bundled with the desktop tier rather than separately: two products confuse, one Pro tier does not.

Cost warning

Conversation sync changes your Firestore bill from negligible to usage-proportional, and it moves user content onto a server for the first time. Encrypt client-side with the existing `crypto.js` account-derived scheme, state plainly in the terms what is stored, and price the tier so a heavy user cannot cost more than they pay.

3.3 B2B and self-host licensing

“An AI client where no conversation, document or API key touches a vendor's server, and which can also drive the machine's filesystem and terminal” is a real procurement pitch for organisations with data-residency obligations: legal, healthcare, finance, public sector, defence suppliers. One site license is worth hundreds of consumer sales.

It is also the slowest path. Before a serious buyer signs you will need single sign-on, an administrative policy layer deciding which tools are enabled organisation-wide, an audit trail, and a response to a security questionnaire. Treat this as a 2027 track that the Phase 1 work happens to make possible, not as something to start now.

3.4 Managed keys and credits

The largest single suppressor of your userbase is the instruction “first, go and get an API key.” Most people who open the app never complete it. Selling prepaid credits, routed through your existing Cloudflare Worker on your own provider account, removes that step. It is also the only model here that scales with *usage* rather than installs.

- **Upside:** converts the drop-off directly into revenue, and improves every other model's funnel.
- **Downside:** it converts a zero-backend app into an operated service. You take on prepaid balance accounting, abuse and fraud risk, provider rate limits, refund handling, and a margin of only 20 to 30 percent over wholesale inference.
- **Verdict:** high value, but the single biggest architectural and legal commitment in this document. Do it after Phase 1 has proven people will pay you at all.

3.5 Search partner revenue

Yogatik already routes `web_search` through DuckDuckGo, Brave, Marginalia and Wikipedia. Several search providers operate syndication or revenue-share programmes that pay a partner client per query; this is how most independent browsers and search extensions are actually funded. It requires no user-visible change and no payment infrastructure.

Unverified: I could not confirm current eligibility criteria, rates, or whether your query volume meets any minimum. Terms change frequently. Confirm directly with each provider's partner programme before assuming any revenue here, and disclose the arrangement in your terms if you enter one.

3.6 Store distribution

The Microsoft Store accepts Electron applications, and a listing costs very little given the installer already exists. Its value is as much *discovery* as revenue, and discovery is your binding constraint. The same logic applies to developer-tool directories and aggregators.

3.7 Low-effort baseline

GitHub Sponsors, a tip jar, and referral links on the provider signup flow you already send users through. None of these will fund development. All cost under a day. Do them because the marginal effort is near zero, not because they are a strategy.

3.8 Explicitly rejected

Several standard tactics for monetizing volume would be actively self-destructive for this particular product. They are listed so the decision is recorded rather than revisited later.

Tactic	Why it is rejected
Reinstating a self-built ad interstitial	Directly caused the existing policy violation. Any full-screen unit whose only content is an ad re-triggers it, at any frequency.
Selling or brokering conversation data or behavioural profiles	Destroys the sole differentiator, and without explicit informed consent is unlawful under the EU GDPR, India's DPDP Act, and the CCPA.
Bundled monetization SDKs that pay per install	Proxyware and adware in practice. Windows Defender and SmartScreen flag them, and an installer that trips antivirus is commercially dead.
Background cryptocurrency mining	Universally classified as malware. Immediate delisting and reputational loss.

Tactic	Why it is rejected
Affiliate-link injection or search-result rewriting	Would add exactly the tracking redirects that research.js currently strips out of results. Hypocritical, and an antivirus and browser-store flag.

The common pattern: each trades the credibility that makes the product worth using for a fraction of a cent per user. It is a poor trade even setting the legal exposure aside.

4. Recommended sequence

Four phases, each with an explicit gate. Do not start a phase before its gate condition is met — the gates exist to stop you building marketplace infrastructure for a userbase that does not exist yet.

Phase	Work	Duration	Gate to proceed
0. Clean up Now	Delete both properties from AdSense. Deploy the ad-free build. Remove ad language from the terms.	1 day	The live build serves no ad code.
1. Charge Weeks 1–4	Desktop license gate, checkout through a merchant of record, pricing page, purchase and activation flow.	2–4 weeks	First ten paying customers, or six weeks elapsed with zero — in which case the pitch is wrong, not the code.
2. Distribute Weeks 3–12	Pre-rendered content surface for search discovery. Store listings. Launch posts in developer communities.	Ongoing	Organic traffic sustained above roughly 5,000 monthly visitors.
3. Scale Month 4 onward	Sync and teams tier, managed credits, marketplace cut, search partner programme.	A quarter or more	Reassess against actual data. Do not pre-commit.

Why charging comes before distributing

It is tempting to grow first and monetize later. Do the opposite. A license gate takes weeks and tells you something no amount of traffic will: whether anyone values this enough to pay. Ten paying users is a stronger signal than ten thousand visitors, and it is far cheaper to discover now than after a quarter of content work.

5. Phase 1 in detail — the desktop license gate

5.1 What is free and what is paid

The split has to feel fair, or the free tier becomes an advertisement for a competitor. Recommendation: everything that makes Yogatik a *good chat app* stays free permanently; everything that makes it a *desktop agent* is paid.

Free (web and desktop)	Pro (desktop only)
All chat, all providers, bring-your-own key	terminal_run and proc_start — real shell execution
All 65 browser-native tools, agents, skills, workflows	The fs_* filesystem tools and per-chat working folders
Documents, retrieval, vision, live voice mode	computer_control — cross-application input
Local models, MCP servers, plugins	browser_control — real tab automation
Encrypted API-key sync across devices	Clipboard history, folder watching, the floating companion

Grandfather every existing user into Pro permanently. It costs nothing at your current size, it is the correct treatment of people who used the product before it had a price, and it produces the reviews that make a store listing work.

5.2 Technical design of the gate

Keep it small. A license check is not a security system; it is a payment reminder for honest people.

Step	Design
Activation	The user pastes a key into a Settings panel. The main process calls the merchant's activation endpoint once, receives a signed activation token, and stores it through the existing safeStorage and DPAPI keychain module, using the kc.v1 sealing path rather than IndexedDB.
Validation	Verify the token's signature locally on every launch. Re-validate against the server every fourteen days. Never block startup on a network call.
Offline grace	Thirty days. A developer on a plane must not lose their filesystem tools. After grace expires, degrade to the free tier with a clear message, never to a crash or a nag loop.
Enforcement point	One function, licensed(), consulted in the renderer's tool registry so that Pro tools are simply absent from the schema list for free users. The agent then never sees them and cannot promise what it cannot do. This is the same pattern the skill and agent tool allowlists already use.
Honest fallback	If a Pro tool is somehow invoked without a license, return the same explicit refusal shape the web build returns for desktop-only tools. Never fail silently.
Piracy	Any client-side check is defeatable. Do not invest in obfuscation. Cracked copies are not lost sales at this scale, and aggressive protection reliably costs more in support burden and bad reviews than it recovers in revenue.

5.3 Choosing a checkout provider

Use a **merchant of record**, not a raw payment gateway. A merchant of record becomes the legal seller and assumes sales-tax liability, which for a solo developer in India selling to EU and US consumers is the largest hidden cost of going paid. EU VAT alone is otherwise a per-country registration problem.

Consideration	What to check before choosing
Merchant of record	Confirm they are the seller of record and handle EU VAT, UK VAT and US sales tax.
License key API	Must issue, activate, validate and revoke keys over HTTP, so that you write no key server.
Payouts to India	Confirm the supported payout method, the currency conversion spread, and any minimum threshold.
Fee structure	Typically around 5% plus payment fees for a merchant of record, versus around 3% for a plain gateway. The difference buys tax compliance and is usually worth it.
Failure mode	If they deprecate the product or ban your account, can you export customers and keys? Check before integrating.

Lemon Squeezy, Polar, Paddle and Gumroad all operate on this model, with different fees and payout support. I have not verified their current fee schedules, India payout support, or present terms — confirm all five points directly before integrating. GST treatment of your own income in India is a question for a chartered accountant, not for this document.

5.4 Pricing

Tier	Price	Rationale
Free	\$0	The full chat client. No time limit and no nagging beyond one unobtrusive Pro mention in Settings.
Pro, one-time	\$39	Includes twelve months of updates, with perpetual use of the version bought. The recommended default.

Tier	Price	Rationale
Pro, annual	\$29/yr	For users who prefer continuous updates. Optional, and do not launch both at once.
Team, 5 seats	\$149	Later, alongside the sync tier.

Launch with one price and one plan. Multiple options at launch produce decision paralysis and support questions, and you have no data yet to justify the segmentation. Consider a launch discount with a real deadline rather than a permanently lower price.

6. Phase 2 in detail — solving distribution

This is the work that the AdSense rejection was a symptom of. It would also make an ad-supported content property viable later, if you ever want one, but its real purpose is user acquisition.

6.1 The technical problem

Firebase Hosting rewrites everything to `index.html`, which contains an empty root element. A crawler indexing `yogatik.web.app` finds one `noscript` paragraph. There is nothing to rank. The fix is not to make the single-page app crawlable; it is to serve genuinely static pages alongside it, which the existing `/platforms` rewrite already demonstrates.

6.2 What to build

- **Static, pre-rendered HTML pages** under a content path, added to `firebase.json` rewrites ahead of the single-page catch-all — exactly the pattern `platforms.html` already uses.
- **Tool documentation**, one page per meaningful capability. Sixty-five tools is sixty-five long-tail search targets, each answering a question someone actually types.
- **Comparison and use-case pages**: running a local model, driving the filesystem from a chat, document question-answering without uploading anything to a vendor.
- **A changelog and release notes**. Cheap to write, updated naturally, and it signals an actively maintained product to both users and search engines.
- **Per-page open-graph images and metadata**. The current site has a single shared icon, so every shared link looks identical.

Quality bar

Thin auto-generated pages will fail exactly the way the app did, and would poison the domain for any future review. Each page must answer a question a real person asked. Twenty genuinely useful pages will outperform two hundred generated ones, and cost far less to maintain.

6.3 Launch channels

Search compounds slowly, so direct launches produce the first users. Developer communities, relevant subreddits, Hacker News and product directories all reward the same thing: a specific, honest claim. Yours is unusually strong and rarely stated — an AI client that runs your shell and your filesystem while sending nothing to a vendor's server. Lead with that, not with the tool count.

7. What to measure

You cannot tune any of this without numbers, and you currently have none. The opt-in analytics module (`analytics.js`, a hard no-op until a sink is supplied, with a key allowlist that drops anything not explicitly permitted) is the right place to add these. It was built so that no message content can leak through it.

Metric	Why it decides something
Monthly active users, web versus desktop	Determines whether penny models are even on the table, and whether the desktop bet is sound.

Metric	Why it decides something
Key-configuration completion rate	Directly sizes the managed-credits opportunity. If most users never finish, that gap is the revenue.
Desktop tool usage: terminal, filesystem, computer control	Validates or refutes the entire Pro split. If nobody uses them, the paid tier is drawn in the wrong place.
Retention at day 7 and day 30	Nothing converts without retention. Fix this before pricing anything.
Free-to-Pro conversion, and where purchases are abandoned	The only number that tunes pricing.

8. Risks, assumptions and open questions

8.1 What I could not verify

- **Your traffic.** No analytics data was available. Every ranking here assumes a small userbase, and materially different numbers would change the order of Section 3.
- **Search partner programme terms.** Eligibility, rates and minimum volumes are unconfirmed and change often. Verify directly before counting on any of it.
- **Checkout provider fees and India payout support.** Confirm current terms with each vendor.
- **Tax treatment.** GST on your income, and the mechanics of receiving foreign payouts, are questions for a qualified accountant. Nothing in this document is tax or legal advice.
- **Policy currency.** My information has a knowledge cutoff. Advertising, store and platform policies change; check the primary sources before acting on any policy claim here.

8.2 Principal risks

Risk	Mitigation
Nobody pays, because the free tier is sufficient	This is the cheapest possible thing to learn, and Phase 1 is designed to learn it within weeks. If the answer is no, the desktop split is wrong, not the idea of charging.
Charging alienates existing users	Grandfather everyone into Pro permanently, keep the free tier genuinely complete, and announce the change before it ships rather than after.
A competitor ships the same desktop capabilities	Likely within a year. Your defence is execution speed and the privacy position, not secrecy — the working code is the differentiator, not the idea.
Provider terms change under managed credits	Do not build Phase 3 credits until Phase 1 revenue justifies the operational exposure, and read the provider terms on reselling before writing any of it.
Support burden exceeds revenue	One-time pricing with a clearly scoped promise, an honest offline grace period, and no aggressive protection keeps support volume proportional to sales.

9. Immediate next actions

#	Action	Owner	When
1	Remove both sites from the AdSense account	You	Today
2	Build and deploy the ad-free version via deploy.bat	Either	Today
3	Decide the free and Pro split in Section 5.1	You	This week
4	Choose and open a merchant-of-record account	You	This week
5	Implement the license gate per Section 5.2	Me, on request	Weeks 1–3
6	Write the first ten content pages per Section 6.2	You	Weeks 2–8

The two items I can start on immediately are the license gate in Section 5.2 and the pre-rendered content surface in Section 6.2. Both are self-contained, and neither blocks the other.